ASSIGNMENT-1

Course: Full Stack Web Development

Course Code: 23CM4121

Student Name: D.Somya

Roll No: A24126552258

Section: B

1. TITLE

Implement Inheritance Types using JavaScript

2. AIM

To demonstrate and implement various types of inheritance in JavaScript, including single, multilevel, hierarchical, and prototypal inheritance using modern ES6 classes and prototype chain syntax.

3. OBJECTIVE

The objectives of this experiment are:

- To understand the concepts of Object-Oriented Programming (OOP) in JavaScript.
- To implement ES6 class-based inheritance using the extends and super keywords.
- To demonstrate Single Inheritance, Multilevel Inheritance, and Hierarchical Inheritance.
- To understand Prototype-based (Prototypal) Inheritance in traditional JavaScript.
- To verify method overriding and property access across parent-child class relationships.

4. THEORY

Inheritance is a fundamental concept of Object-Oriented Programming (OOP) that allows a child class to acquire properties and methods from a parent class, promoting code reusability.

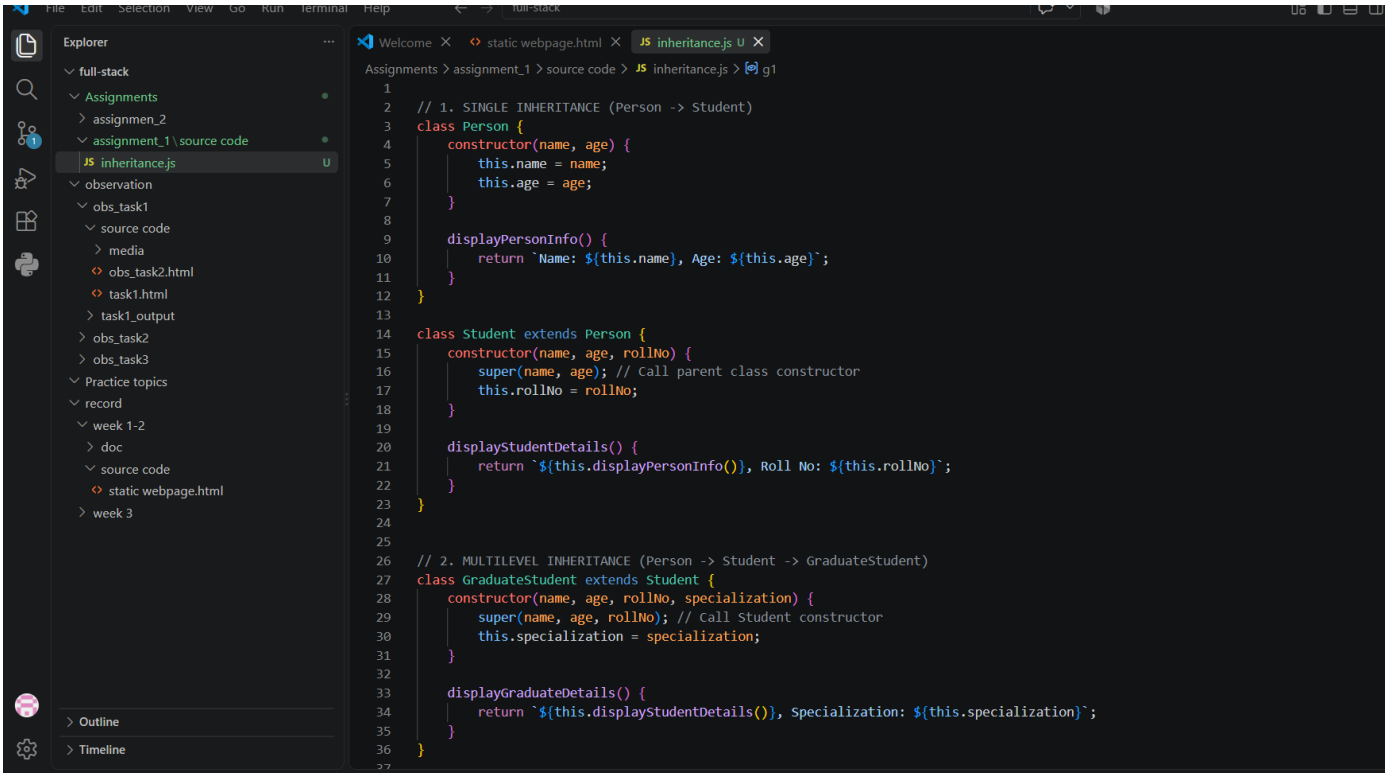
Inheritance Types Implemented:

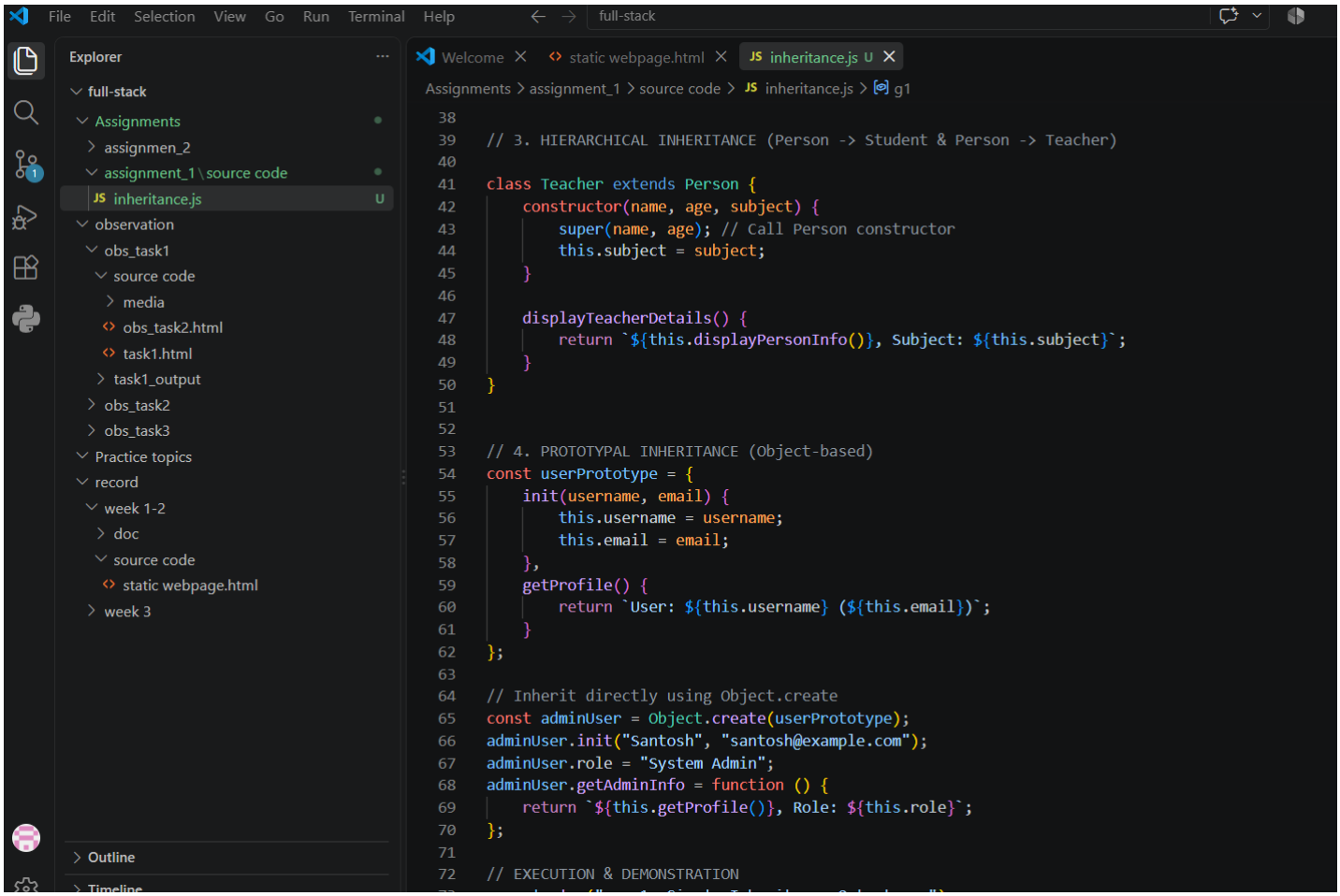
1. **Single Inheritance:** A single child class inherits directly from a single parent class.
2. **Multilevel Inheritance:** A child class inherits from a parent class, which itself inherits from another superclass forming a chain.
3. **Hierarchical Inheritance:** Multiple child classes inherit directly from a single parent class.
4. **Prototypal Inheritance:** An object dynamically links to another prototype object to share methods without using class abstractions.

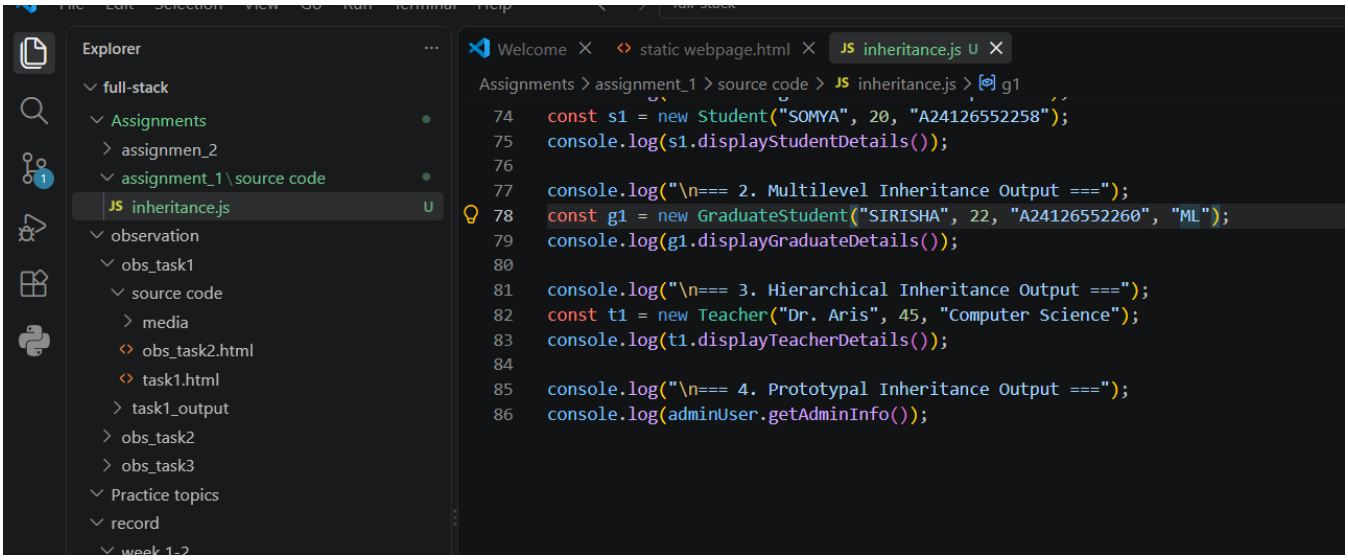
5. ALGORITHM / PROCEDURE

1. Create a main JavaScript script file named inheritance.js.
2. **Single Inheritance:** Define a parent class Person with a constructor and method. Create a child class Student that uses extends Person and calls super().
3. **Multilevel Inheritance:** Create a derived class GraduateStudent that extends Student, creating a three-tier class chain (Person → Student → GraduateStudent).
4. **Hierarchical Inheritance:** Create another child class Teacher that extends the base class Person directly alongside Student.
5. **Prototypal Inheritance:** Create a base prototype object using standard object literals and derive a new object using Object.create().
6. Instantiation: Instantiate objects for each class/prototype type.
7. Execute inherited and overridden methods on each object instance to check behavior.
8. Output the results using console.log() statements.
9. Run the script using Node.js (node inheritance.js) or execute inside a web browser developer console.
10. Validate actual execution outputs against expected outputs across all inheritance test cases.

6. PROGRAM



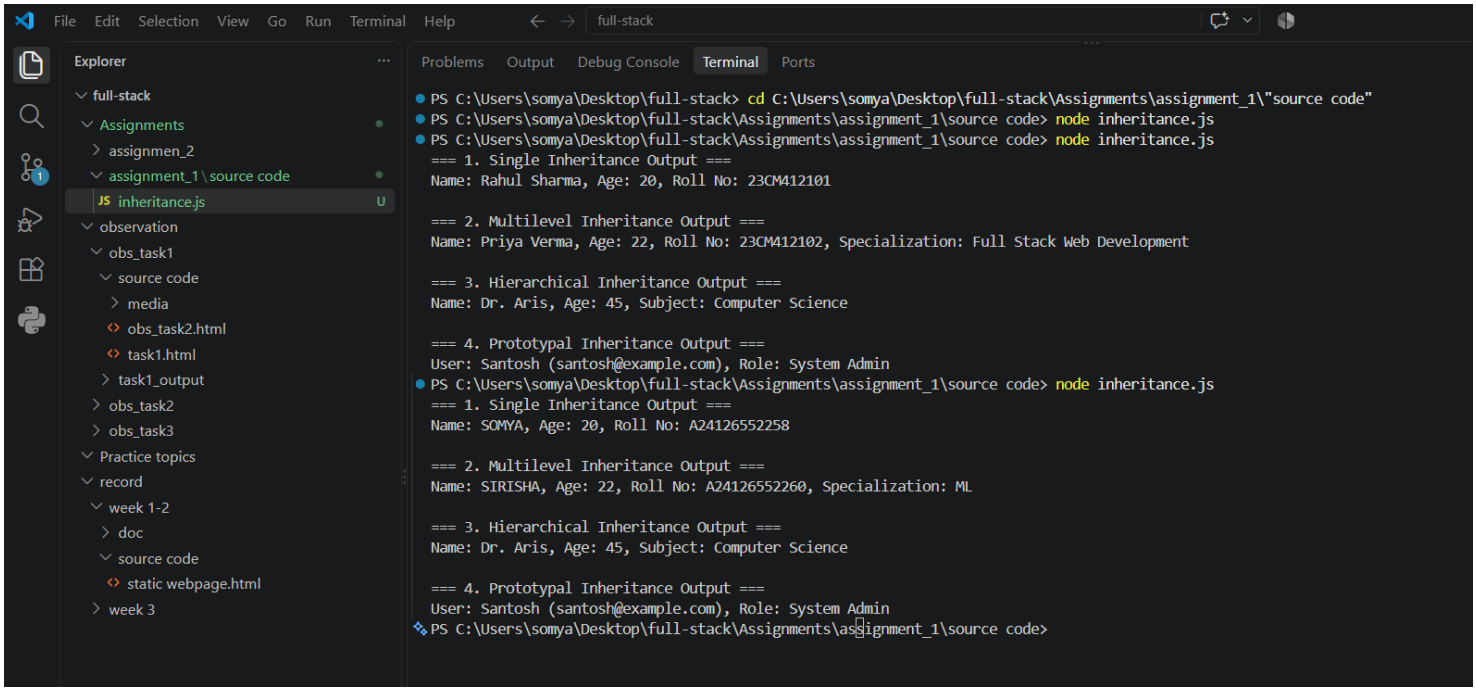

File Edit Selection View Go Run Terminal Help
full-stack
Explorer
full-stack
Assignments
assignmen_2
assignment_1 \ source code
inheritance.js U
observation
obs_task1
source code
media
obs_task2.html
task1.html
task1_output
obs_task2
obs_task3
Practice topics
record
week 1-2
doc
source code
static webpage.html
week 3
Outline
Timeline
Welcome
static webpage.html
inheritance.js U
Assignments > assignment_1 > source code > inheritance.js > g1
38
39 // 3. HIERARCHICAL INHERITANCE (Person -> Student & Person -> Teacher)
40
41 class Teacher extends Person {
42 constructor(name, age, subject) {
43 super(name, age); // Call Person constructor
44 this.subject = subject;
45 }
46
47 displayTeacherDetails() {
48 return `${this.displayPersonInfo()}, Subject: ${this.subject}`;
49 }
50 }
51
52
53 // 4. PROTOTYPAL INHERITANCE (Object-based)
54 const userPrototype = {
55 init(username, email) {
56 this.username = username;
57 this.email = email;
58 },
59 getProfile() {
60 return `User: ${this.username} (${this.email})`;
61 }
62 };
63
64 // Inherit directly using Object.create
65 const adminUser = Object.create(userPrototype);
66 adminUser.init("Santosh", "santosh@example.com");
67 adminUser.role = "System Admin";
68 adminUser.getAdminInfo = function () {
69 return `${this.getProfile()}, Role: ${this.role}`;
70 };
71
72 // EXECUTION & DEMONSTRATION


Explorer
full-stack
Assignments
assignmen_2
assignment_1 \ source code
inheritance.js U
observation
obs_task1
source code
media
obs_task2.html
task1.html
task1_output
obs_task2
obs_task3
Practice topics
record
Welcome
static webpage.html
inheritance.js U
Assignments > assignment_1 > source code > inheritance.js > g1
74 const s1 = new Student("SOMYA", 20, "A24126552258");
75 console.log(s1.displayStudentDetails());
76
77 console.log("\n=== 2. Multilevel Inheritance Output ===");
78 const g1 = new GraduateStudent("SIRISHA", 22, "A24126552260", "ML");
79 console.log(g1.displayGraduateDetails());
80
81 console.log("\n=== 3. Hierarchical Inheritance Output ===");
82 const t1 = new Teacher("Dr. Aris", 45, "Computer Science");
83 console.log(t1.displayTeacherDetails());
84
85 console.log("\n=== 4. Prototypal Inheritance Output ===");
86 console.log(adminUser.getAdminInfo());

7. CODE EXPLANATION

Code / Keyword	Explanation
class ClassName	Defines an ES6 blueprint template for creating objects.
extends	Keyword used in class declarations to create a child class of another parent class.
super()	Calls the parent class constructor to initialize inherited properties inside the child class.
Person	Base class containing common properties (name, age) and generic method displayPersonInfo().
Student extends Person	Implements Single Inheritance by extending properties and methods of Person.
GraduateStudent extends Student	Implements Multilevel Inheritance by extending Student, inheriting from both Student and Person.
Teacher extends Person	Implements Hierarchical Inheritance by creating a second independent child branch off Person.
Object.create	Creates a new object linked to proto as its prototype, achieving native Prototypal Inheritance. Student extends Person

8. OUTPUT



s

9.RESULT

Thus, various types of inheritance including Single, Multilevel, Hierarchical, and Prototypal Inheritance were successfully implemented and verified in JavaScript